

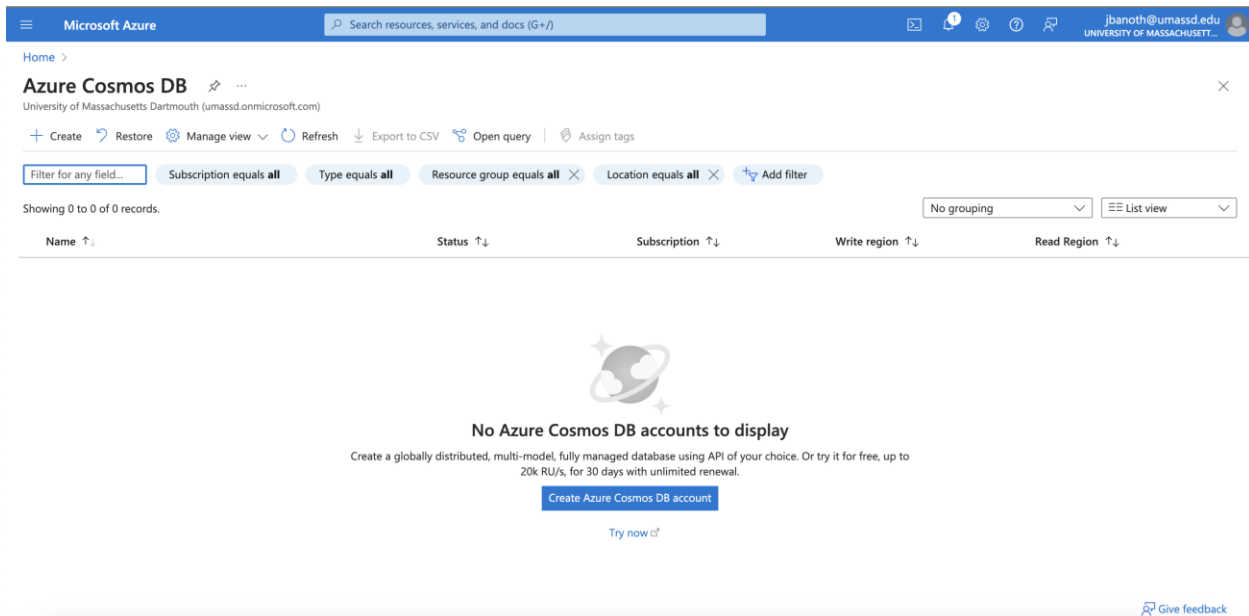
Jeevan Kumar Banoth – 02105145

CIS 552: Database Design

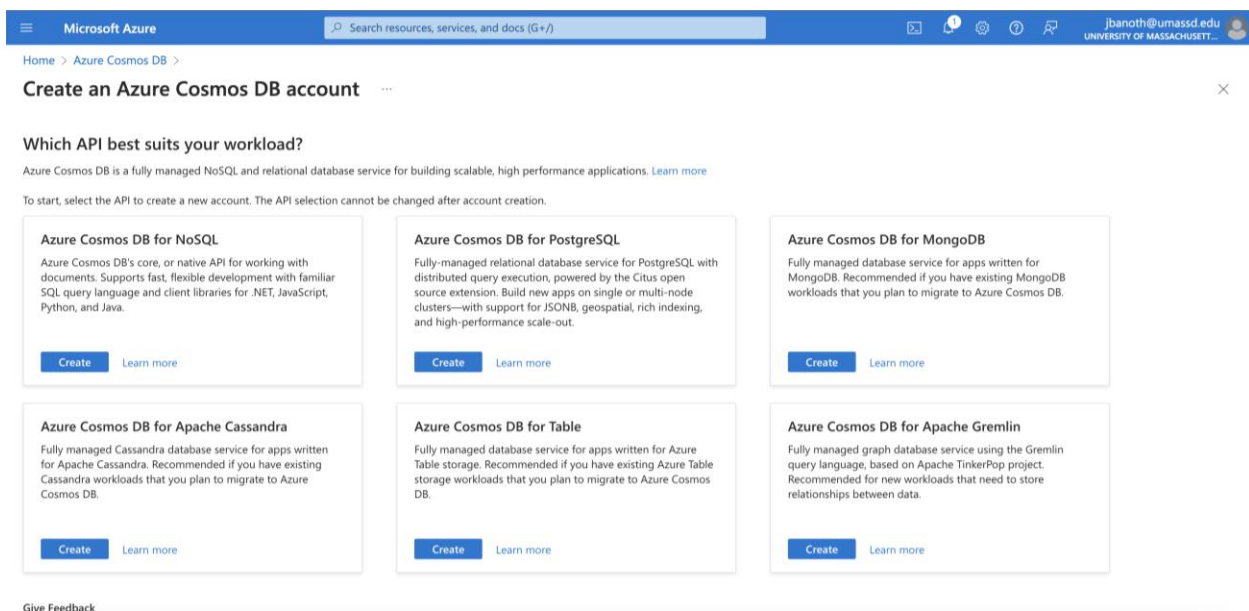
Homework – 5:

Step 1: Amongst other things, we need to create an account for Microsoft Azure Cosmos DB

From the below image which shows that Azure Cosmos DB has been created, there is no DB hence the need for a new one



This stage requires us to choose Azure Cosmos DB for NoSQL and to continue with additional information.



Now, we will be creating the Cosmos DB for NoSQL as below and next, review and creat then we will have the Cosmos Database ready in some time.

Home > Azure Cosmos DB >

Create Azure Cosmos DB Account - Azure Cosmos DB for NoSQL

Project Details
Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * ▼ Azure subscription 1

Resource Group * ▼ (New) Umass_DB
[Create new](#)

Instance Details

Account Name * ✓ jeevanrishi

Configure availability zone settings for your account. You cannot change these settings once the account is created.

Availability Zones ⓘ ☐ Enable ☒ Disable

Location * ⓘ ▼ (US) Central US

Available locations are determined by your subscription's access and availability zone support (if that is enabled). If you don't see or cannot select your desired location, please open a support request for region access.
[Click here for more details on how to create a region access request](#)

Capacity mode ⓘ ☒ Provisioned throughput ☐ Serverless
[Learn more about capacity mode](#)

With Azure Cosmos DB free tier, you will get the first 1000 RU/s and 25 GB of storage for free in an account. You can enable free tier on up to one account per subscription. Estimated \$64/month discount per account.

Apply Free Tier Discount ☒ Apply ☐ Do Not Apply

Limit total account throughput ☒ Limit the total amount of throughput that can be provisioned on this account

ⓘ This limit will prevent unexpected charges related to provisioned throughput. You can update or remove this limit after your account is created.

[Review + create](#) [Previous](#) [Next: Global distribution](#) [Feedback](#)

This below Image is the deployment of one, it will take some time and then we are ready to go.

Home >

Microsoft.Azure.CosmosDB-20240723151711 | Overview

Deployment

Search × ◀ ▶ 🗑️ Delete ⏸️ Cancel 🔄 Redeploy ⬇️ Download 🔄 Refresh

Overview

- Inputs
- Outputs
- Template

*** Deployment is in progress

Deployment name : Microsoft.Azure.CosmosDB-20240723151711 Start time : 7/23/2024, 3:17:13 PM
Subscription : Azure subscription 1 Correlation ID : 6e94fd79-eaed-4af0-951e-8397204e89dc
Resource group : Umass_DB

▼ Deployment details

Resource	Type	Status	Operation details
jeevanrishi	Microsoft.DocumentDb/databases	OK	Operation details

Give feedback
[Tell us about your experience with deployment](#)

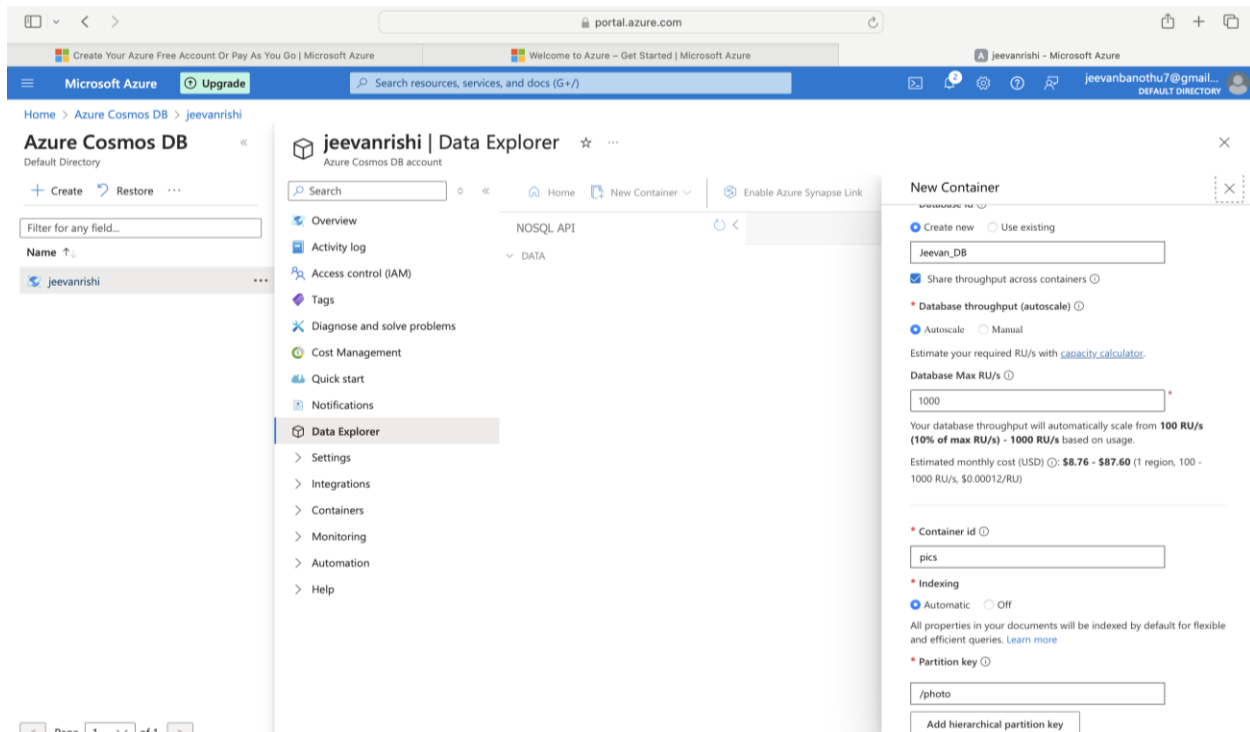
Microsoft Defender for Cloud
Secure your apps and infrastructure
[Go to Microsoft Defender for Cloud >](#)

Free Microsoft tutorials
[Start learning today >](#)

Work with an expert
Azure experts are service provider partners who can help manage your assets on Azure and be your first line of support.
[Find an Azure expert >](#)

Step 2: Creating a Sample Container:

The container is somewhat like a SQL database table and saves JSON documents. We have set the container identifier “music,” which can symbolize a group of music-related files, and a partition key “/artist,” which divides the data into various parts for scalability.



When you are selecting a partition key, it is essential to select an attribute which has a large variety of values and frequently acts as a filter in the queries.

This ensures that information is evenly spread among departments and makes queries as well as transactions operate more efficiently. If your queries usually use 'player' as one of their filters while your data set contains many different players, then this could be good candidate for disk key.

Step 3: Establishing Container Entities:

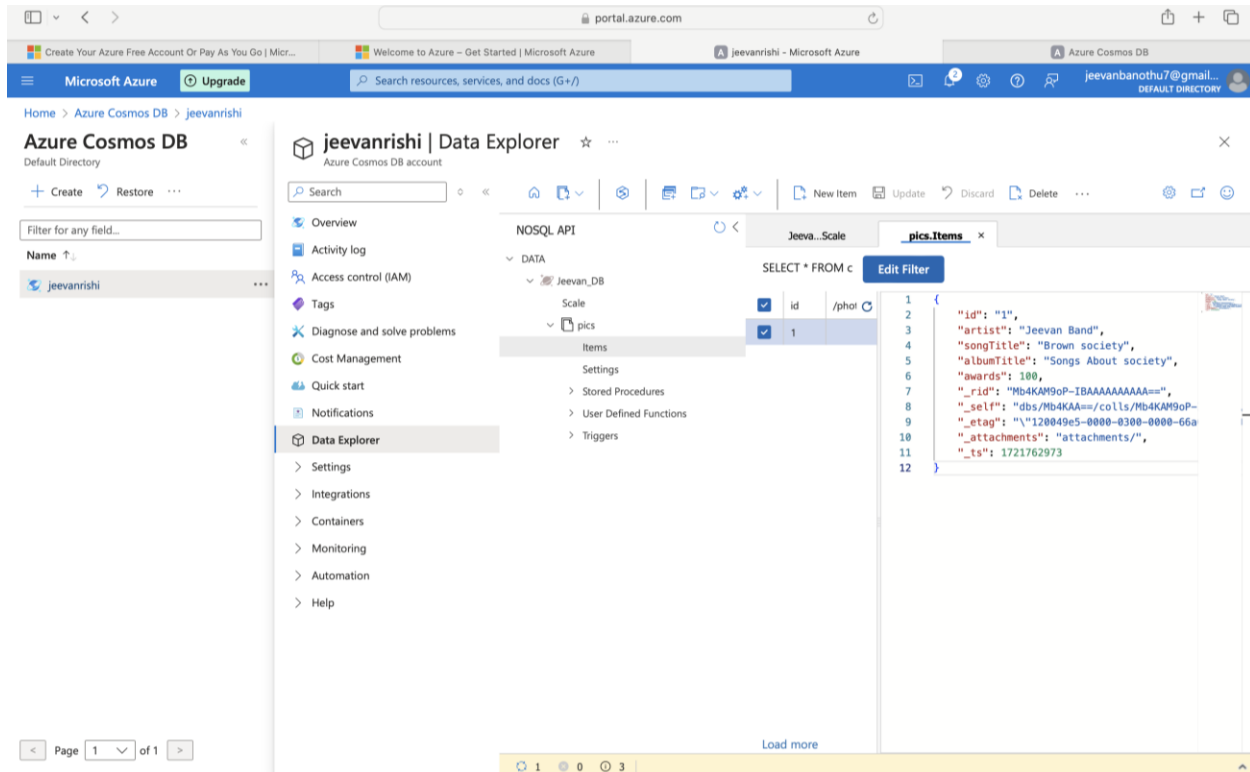
Currently, we are using the insert tab to add additional elements to the music.

id: A distinct identification.

artist: "Jeevan Band" is the name of the artist.

song title: "Brown Society" is the song's title.

"Songs About Society" is the title of the album.



honors: The song has received ten accolades overall.

id: The container's unique identifier, which is set to "1".

Artist: The name of the artist (in this example, "Arijit Singh") connected to the query.

SongTitle: The song's title, which is "Chaiyya" in this case.

Fifteen awards have been given to this song or performer.

removed Resource ID: a unique internal number assigned to each resource in Azure Cosmos DB

self: The Azure Cosmos DB resource's Uniform Resource Locator (URI), which is used to obtain the resource via the REST API.

An eTag is an entity tag used for optimistic concurrency control. Users are less likely to inadvertently overwrite changes made by other users.

_attachments: This is only a placeholder for where any attachments related to the query may be found.

_ts: The most recent update is shown by this timestamp. The number 1711499141 denotes the number of seconds that have elapsed since the Unix epoch, which occurred on January 1, 1970.

Explorer ☆ ...

NOSQL API

DATA

- Jeevan_db
 - Scale
 - Music
 - Items
 - Settings
 - Stored Procedures
 - User Defined Functions
 - Triggers

Home Music.Items x

SELECT * FROM c Edit Filter

	id	/Artist
☐	1	Acm...
☐	2	Omi
☐	3	Es Jay
☐	4	Arijt...
☐	5	Diljit...
☐	6	XXX...
☐	7	Gee...
☐	8	Jeev...
☐	9	Blac...
☐	10	Cha...
☐	11	Van...
☐	12	Raj
☐	13	asm...
☐	14	Dk
☒	15	Kohli

Load more

```

1 {
2   "id": "15",
3   "Artist": "Kohli",
4   "songTitle": "King",
5   "albumTitle": "Cricket",
6   "awards": 100000,
7   "_rid": "4XRhAMSDcgISAAAAAAAAA==",
8   "self": "dbs/4XRhAA=/colls/4XRhAMSDcgI=/docs/4XRhAMSDcgISAAAAAAAAA==/",
9   "etag": "\"2500732d-0000-0300-0000-66a00c290000\"",
10  "_attachments": "attachments/",
11  "_ts": 1721764905
12 }

```

0 1 21

Lines, they are as diverse as love letters, representing documents within the container. Accompanied by a unique ID as well as sender names, each line corresponds with a specific document in the collection.

The artist connected to a text with document ID of 3 is “Arijit Singh”. This view is sorted by identifiers of individual documents which appear like in the table below either being automatically or manually allocated such sequential numbers for distinguishing them from one another.

Step:4: Querying:

The screenshot shows the Azure Data Explorer interface. The left sidebar contains a navigation menu with options like Overview, Activity log, Access control (IAM), Tags, Diagnose and solve problems, Cost Management, Quick start, Notifications, and Data Explorer. The main pane displays the 'Music' container under the 'Jeevan_db' database. A query is executed: `SELECT * FROM c WHERE c.Artist = 'Jeevan'`. The results pane shows a single document with the following JSON structure:

```
{
  "id": "8",
  "Artist": "Jeevan",
  "songTitle": "Box office",
  "albumTitle": "Lost",
  "awards": 299,
  "_rid": "4XRhAMSDcgILAAAAAAAAA==",
  "_self": "dbs/4XRhAA=/colls/4XRhAMSDcgI=/docs/4XRhAMSDcgILAAAAAAAAA==/",
  "_etag": "\"2500f329-0000-0300-0000-66a0b240000\"",
  "_attachments": "attachments/",
  "_ts": 1721764644
}
```

A status bar at the bottom indicates 'Successfully fetched 1 item for container Music'.

A QL-like query is invoked to extract every document from ‘the music bin’ (or ‘c’), in which the artist field has ‘Jeevan’ as its value. id: Unique identifier for this document belonging to that container; value “8”. Artist: An artist’s name is given as “Jeevan”. Song Title: Title of the song, i.e., “Box office”. The number of awards listed is 299.

The screenshot shows the Azure Data Explorer interface. The left sidebar contains a navigation menu with options like Overview, Activity log, Access control (IAM), Tags, Diagnose and solve problems, Cost Management, Quick start, Notifications, and Data Explorer. The main pane displays the 'Music' container under the 'Jeevan_db' database. A query is executed: `SELECT * FROM c`. The results pane shows multiple documents with the following JSON structure:

```
{
  "id": "1",
  "artist": "Acme Band",
  "songTitle": "Happy Day",
  "albumTitle": "Songs About Life",
  "awards": 10,
  "_rid": "4XRhAMSDcgIBAAAAAAAAA==",
  "_self": "dbs/4XRhAA=/colls/4XRhAMSDcgI=/docs/4XRhAMSDcgIBAAAAAAAAA==/",
  "_etag": "\"2500151b-0000-0300-0000-66a00750000\"",
  "_attachments": "attachments/",
  "_ts": 1721763957
},
{
  "id": "1",
  "Artist": "Acme Band",
  "songTitle": "Happy Day",
  "albumTitle": "Songs About Life",
  "awards": 10,
  "_rid": "4XRhAMSDcgIBAAAAAAAAA==",
  "_self": "dbs/4XRhAA=/colls/4XRhAMSDcgI=/docs/4XRhAMSDcgIBAAAAAAAAA==/",
  "_etag": "\"2500151b-0000-0300-0000-66a00750000\"",
  "_attachments": "attachments/",
  "_ts": 1721763957
}
```

A status bar at the bottom indicates 'Successfully fetched 16 items for container Music'.

Every field in the documents must be returned, according to a SQL-like query that gets every document from the sample database music box.

ID: "1"

Artist: "Artist Name" Song title: "Song Title" Album title: "Album Title" Issued in 2020

Category: "Genre"

The screenshot shows the Azure Data Explorer web interface. The left sidebar contains navigation options like Overview, Activity log, Access control (IAM), Tags, Diagnose and solve problems, Cost Management, Quick start, Notifications, and Data Explorer. The main pane displays a query in the NOSQL API editor. The query is: `SELECT * FROM c ORDER BY c.timestamp DESC`. Below the query, the 'Results' tab shows a list of 16 documents. The first document is a JSON object with fields: id, Artist, songTitle, albumTitle, awards, _rid, _self, _etag, _attachments, and _ts. The second document is also a JSON object with fields: id, Artist, songTitle, albumTitle, and awards.

```
1 SELECT * FROM c ORDER BY c.timestamp DESC
2
```

Results Query Stats

1 - 16

```
{
  "id": "15",
  "Artist": "Kohli",
  "songTitle": "King",
  "albumTitle": "Cricket",
  "awards": 100000,
  "_rid": "4XRhAMSDcgISAAAAAAAAA==",
  "_self": "dbs/4XRhAA=/colls/4XRhAMSDcgISAAAAAAAAA==/",
  "_etag": "\"2500732d-0000-0300-0000-66a00c290000\"",
  "_attachments": "attachments/",
  "_ts": 1721764905
},
{
  "id": "14",
  "Artist": "DK",
  "songTitle": "opinion",
  "albumTitle": "Car",
  "awards": 123,

```

You can order documents based on a given field using the Cosmos DB Core (SQL) API, which often has query syntax like SQL. Here, c stands for the container's alias, and timestamp is a required field in all documents that contains the creation or last update time.

The screenshot shows the Azure Data Explorer interface. The left sidebar contains navigation options like Overview, Activity log, Access control (IAM), Tags, Diagnose and solve problems, Cost Management, Quick start, Notifications, and Data Explorer. The main pane displays a query in the NOSQL API: `SELECT * FROM c WHERE ARRAY_CONTAINS(c['Award Years'], 2009)`. The results pane shows a single document with the following structure:

```
{
  "id": "3",
  "Artist": "Es Jay",
  "songTitle": "Brown society",
  "Award Years": [
    2008,
    2009
  ],
  "albumTitle": "People",
  "awards": 10,
  "_rid": "4XRhAMSDcgIGAAAAAAAAA==",
  "_self": "dbs/4XRhAA=/colls/4XRhAMSDcgI=/docs/4XRhAMSDcgIGAAAAAAAAA=/",
  "_etag": "\"2500c824-0000-0300-0000-66a009cb0000\"",
  "_attachments": "attachments/",
  "_ts": 1721764299
}
```

The status bar at the bottom indicates "Successfully fetched 4 item for container Music".

Checks whether the array c ['Award Years'] has the value 2009 using the ARRAY_CONTAINS function.

The artist "Taylor Swift" and the track "Love Story" are identified by the viewable document id, which is "1". This song won awards in both years, included in the award year table, 2009 and 2010.

The screenshot shows the Azure Data Explorer interface with a query: `SELECT * FROM c ORDER BY c.songTitle`. The results pane displays a list of documents. The first document (id "1") is:

```
{
  "id": "1",
  "Artist": "Taylor Swift",
  "songTitle": "Love Story",
  "albumTitle": "Fearless",
  "awards": 200,
  "_rid": "4XRhAMSDcgIGAAAAAAAAA==",
  "_self": "dbs/4XRhAA=/colls/4XRhAMSDcgI=/docs/4XRhAMSDcgIGAAAAAAAAA=/",
  "_etag": "\"2500c824-0000-0300-0000-66a009cb0000\"",
  "_attachments": "attachments/",
  "_ts": 1721764299
}
```

The status bar at the bottom indicates "Successfully fetched 16 item for container Music".

For papers with IDs less than 4, the complete paragraph name field is displayed; for documents with IDs greater than 4, only the ID field is displayed. Take "Love Story" as an example, denoted by id 4, and "Bad Romance" by id 2.

Conclusion:

In this lab, we deftly maneuvered around setting up and using an entirely managed NoSQL database service known as Microsoft Azure Cosmos DB. The lab took us through how to use Cosmos DB beginning with the creation of a Cosmos DB account.

Upon initialization, we spotted a section asking for a new database instance to be created. Thereafter, we moved ahead to create a music container within our sample database with "music" as its container ID and "/artist" as its section.

The key This is meant to ensure that both queries and transactions perform better since they will not always be directed towards the same set of data; it also signifies the heightening dimensions of scalability within Cosmos DB.

When consuming data from the music repository, we leveraged on the flexible model of Cosmos DB. adapt to different musical entities as an artist. name, canton name, album name, awards, and year of release.

These characteristics show off the schema-free nature of Cosmos DB which allows us to manage diverse data architectures in one container.

By executing a sequence of SQL like commands on basic datasets we illustrated how powerful examples for querying functions are using some queries in CosmosDB which are mainly limited by category-based filters or even personalized timestamps available through sorting results alphabetically.

Overall, this interactive lab gave participants a thorough understanding of Azure Cosmos DB and demonstrated how well it manages NoSQL data in addition to having a strong query engine that enables scalable, dependable support for intricate tasks.